

ТИТУЛЬНЫЙ ЛИСТ

По дисциплине пм 01 мдк 01 мдк 02

Разветртыване LLM сервера для разработки

СОДЕРЖАНИЕ

Титульный лист.....	1
Содержание.....	2
Термины и определения.....	3
Перечень сокращений и обозначений.....	4
Введение.....	5
1 Анализ решений для локального развёртывания LLM и обоснование выбора llama.cpp.....	7
1.1 Ollama.....	7
1.2 Llama.cpp.....	8
1.3 Вывод анализа.....	9
2 Определение аппаратных требований к серверной платформе.....	10
2.1 Факторы, определяющие аппаратные требования.....	10
3 РАЗРАБОТКА КОНФИГУРАЦИИ СЕРВЕРА.....	11
3.1 Подготовка программного окружения.....	11
3.2 Скрипт запуска сервера.....	11
3.3 Работа модели.....	13
4 РЕАЛИЗАЦИЯ ИНТЕГРАЦИИ ЛОКАЛЬНОГО LLM-СЕРВЕРА СО СРЕДОЙ РАЗРАБОТКИ CODE OSS.....	14
4.1 Выбор инструмента интеграции: расширение Continue.....	14
4.2 Проверка работоспособности интеграции.....	15
4.3 Выводы по интеграции.....	17
Заключение.....	18
Список использованных источников.....	19

ТЕРМИНЫ И ОПРЕДЕЛЕНИЯ

В настоящей курсовой работе применяют следующие термины с соответствующими определениями:

API (Application Programming Interface) - программный интерфейс, то есть описание способов взаимодействия одной компьютерной программы с другими

AI - также ИИ, искусственный разум, в самом широком смысле — научно-технологическая область, занимающаяся изучением и созданием интеллектуальных сущностей

LLM (large language model) - Большая языковая модель основанная на нейронной сети с множеством параметров (миллиарды весовых коэффициентов и более), которая проходит предварительное обучение на обширных массивах размеченного текста методами самообучения

ПЕРЕЧЕНЬ СОКРАЩЕНИЙ И ОБОЗНАЧЕНИЙ

В настоящей курсовой работе применяют следующие сокращения и обозначения:

ВВЕДЕНИЕ

Современные IT-компании активно внедряют инструменты искусственного интеллекта в процессы разработки. Согласно исследованию Stack Overflow 2025 года, 84% разработчиков используют или планируют использовать AI-инструменты, при этом 51% применяют их ежедневно. Исследование Complexity Science Hub показывает, что доля AI-ассистированного кода на GitHub выросла с 5% в 2022 году до 29% к концу 2024 года.

Однако рост популярности облачных AI-решений (GitHub Copilot, ChatGPT) обостряет проблему безопасности интеллектуальной собственности. Передача фрагментов кодовой базы сторонним серверам создаёт риски утечки конфиденциальных алгоритмов и коммерческой тайны. Эксперты отмечают, что при использовании облачных AI-инструментов возникают: размывание границ суверенитета данных, неконтролируемость цепочек вывода и расширение поверхности атаки.

Актуальность работы определяется тремя факторами. Во-первых, необходимостью защиты кодовой базы: облачные AI-сервисы могут использовать переданный код для дообучения моделей, что создаёт риски раскрытия интеллектуальной собственности. Во-вторых, потребностью в автономности разработки при ограниченном доступе в Интернет или работе в закрытых периметрах. В-третьих, экономической целесообразностью: локальное развёртывание позволяет избежать регулярных абонентских платежей.

Существует ряд open-source инструментов для локального запуска моделей (Ollama, llama.cpp, vLLM). Однако проблема системного подхода к интеграции такого сервера в реальную рабочую среду организации, с учётом требований администрирования и удобства для конечных пользователей, остаётся недостаточно проработанной.

Объект и предмет исследования:

- 1) Объект исследования: процесс организации безопасной локальной инфраструктуры для использования LLM в разработке ПО.
- 2) Предмет исследования: методы развёртывания, администрирования и интеграции локального LLM-сервера (на базе llama-server) в рабочую среду IT-специалистов, в частности - подключение к Code OSS.

Цель работы: разработать и экспериментально подтвердить работоспособность локального LLM-сервера на базе llama-server, обеспечить его администрирование в условиях локальной сети и выполнить интеграцию со средой разработки Code OSS.

Задачи:

- 1) Провести анализ существующих решений для локального развёртывания LLM и обосновать выбор llama-server.
- 2) Определить минимальные аппаратные требования и выполнить установку и настройку llama-server в локальной сети.
- 3) Разработать конфигурацию сервера, обеспечивающую стабильную работу.
- 4) Реализовать интеграцию локального LLM-сервера с Code OSS на клиентском компьютере.
- 5) Выполнить экспериментальную проверку работоспособности решения .
- 6) Сформулировать рекомендации по администрированию сервера .

1 АНАЛИЗ РЕШЕНИЙ ДЛЯ ЛОКАЛЬНОГО РАЗВЁРТЫВАНИЯ LLM И ОБОСНОВАНИЕ ВЫБОРА LLAMA.CPP

Для организации локального LLM-сервера, доступного нескольким пользователям в корпоративной сети, наиболее актуальными являются два решения: Ollama и Llama.cpp (с компонентом llama-server). Оба инструмента базируются на единой технологической основе (формат GGUF, оптимизации CPU/GPU), но принципиально различаются по архитектуре и подходу к администрированию. // вставить источники на сравнение ollama vs llama.cpp

1.1 Ollama

Ollama позиционируется как «LLM-движок для разработчиков», предлагая максимально простой путь запуска моделей. Однако при использовании в многопользовательском сценарии проявляются его некоторые недостатки

Достоинства:

- 1) Установка одной командой, готовые пакеты для всех ОС
- 2) Встроенный реестр моделей
- 3) Open-AI совместимый API

Недостатки:

- 1) Последовательная обработка запросов — при одновременных обращениях нескольких пользователей запросы ставят в очередь
- 2) Ограниченный контроль над ресурсами — администратор не может распределить потоки CPU между задачами
- 3) Отсутствие встроенного веб-интерфейса

- 4) Логирование и мониторинг — стандартные средства логирования недостаточны для реальной эксплуатации

1.2 Llama.cpp

Llama-server — это встроенный HTTP-сервер, входящий в состав проекта Llama.cpp, предназначенный для production-развёртывания языковых моделей в корпоративной среде. В отличие от Ollama, который ориентирован на индивидуальных разработчиков и простоту использования, Llama-server предоставляет администраторам организации полный спектр инструментов для управления многопользовательской инфраструктурой.

Достоинства:

- 1) Параллельная обработка сессий — поддержка конкурентных сессий с возможностью ограничения количества одновременных сессий
- 2) Полный контроль над ресурсами — управление количеством потоков CPU приоритетами, размером контекстного окна для каждого пользователя.
- 3) Встроенный веб-интерфейс позволяет наблюдать за работой сервера, просматривать историю запросов без дополнительных надстроек
- 4) Детальное логирование — все запросы и ответы логируются с возможностью настройки уровня детализации
- 5) Anthropic-совместимый API — расширяет возможности интеграции с различными клиентами.

1.3 Вывод анализа

Ollama представляет собой удобное решение для индивидуального использования или малых групп разработчиков, где допустима последовательная обработка запросов. Однако в сценарии корпоративного локального LLM-сервера, обслуживающего нескольких пользователей одновременно, его архитектурные ограничения становятся критическими.

llama-server обеспечивает необходимый уровень контроля, масштабируемости и наблюдаемости, что делает его оптимальным выбором в качестве базового компонента для развёртывания локального LLM-сервера в организации. Более высокая сложность первичной настройки компенсируется гибкостью и предсказуемостью работы в многопользовательской среде.

2 ОПРЕДЕЛЕНИЕ АППАРАТНЫХ ТРЕБОВАНИЙ К СЕРВЕРНОЙ ПЛАТФОРМЕ

Для успешного развёртывания локального LLM-сервера на базе llama.cpp необходимо определить минимальные аппаратные требования, обеспечивающие стабильную работу в многопользовательской среде. В отличие от индивидуального использования, где допустимы периодические задержки, серверная инфраструктура организации должна гарантировать предсказуемое время отклика при одновременной работе нескольких пользователей.

2.1 Факторы, определяющие аппаратные требования

Характеристики модели — основной объём потребляемой памяти зависит от выбранной модели и типа квантования. Однако при загрузке в оперативную память требуется дополнительное пространство для KV-кэша, размер которого пропорционален контекстному окну и количеству параллельных сессий.

Многопользовательская нагрузка — при параллельной обработке запросов от нескольких пользователей требования к памяти, GPU и CPU возрастают пропорционально количеству одновременных сессий. Каждый параллельный слот требует выделения собственного KV-кэша.

Размер контекстного окна — увеличение контекстного окна существенно повышает потребление памяти. Зависимость линейная: при удвоении контекстного окна вдвое возрастает размер KV-кэша на каждую сессию.

2.2 Требования к ОЗУ

Общий объём оперативной памяти, необходимый для работы Llama-server в многопользовательской среде, складывается из трёх основных компонентов.

Общий объём оперативной памяти для Llama-server складывается из трёх составляющих.

Размер модели — определяется количеством параметров и типом квантования. Например, модель 7B в Q4_K_M занимает ~4 ГБ, а в Q8_0 — уже ~7 ГБ. Чем выше точность квантования, тем больше памяти требуется.

KV-кэш — выделяется для каждой параллельной сессии и хранит промежуточные состояния токенов контекстного окна. Его размер зависит от архитектуры модели, длины контекста и количества одновременных сессий. При увеличении контекстного окна с 2048 до 8192 токенов потребление памяти на сессию растёт в четыре раза. Каждая новая параллельная сессия добавляет свой KV-кэш.

Служебные расходы — буферы, очереди и управляющие структуры, обычно составляющие 5-10% от основного объёма.

Таким образом, итоговое потребление памяти всегда больше простого размера модели. Для модели 7B с контекстом 4096 токенов и тремя параллельными сессиями требуется 8-12 ГБ, для модели 13B при тех же условиях — 16-20 ГБ.

Для одного пользователя модели 7B достаточно 6-8 ГБ ОЗУ. Для трёх-четырёх разработчиков — уже 16 ГБ. Для модели 13B и пяти пользователей рекомендуется 32 ГБ. Также стоит учитывать, что длинные ответы увеличивают заполнение KV-кэша, что дополнительно повышает потребление памяти.

2.3 Требования к CPU и GPU

Plama.cpp эффективно работает как на x86_64, так и на AArch64 архитектурах. При использовании CPU для вычислений критическое значение имеет количество ядер и поддержка векторных инструкций (AVX2, AVX-512 для Intel/AMD или NEON для ARM).

Для многопользовательских сценариев на серверах с большим количеством ядер рекомендуется использовать CPU affinity для привязки процессов к конкретным ядрам, а так же NUMA-aware оптимизации для работы с неоднородной памятью.

При использовании GPU для ускорения вычислений (с параметром -ngl) достаточно 2-4 ядер CPU для управления передачей данных между памятью и GPU. Основная нагрузка ложится на видеокарту, поэтому критическими становятся объём VRAM и пропускная способность памяти.

2.4 Требования к дисковому пространству

Дисковое пространство необходимо для хранения файлов моделей и логов сервера:

Файлы моделей — вес зависит от количества параметров и типа квантования:

- 1) Модель 7B в Q4_K_M: ~3.5-4 ГБ;
- 2) Модель 13B в Q4_K_M: ~7-8 ГБ;
- 3) Модель 70B в Q4_K_M: ~35-40 ГБ.

Логирование — рекомендуется выделить 5-10 ГБ для хранения истории запросов и системных логов.

Системные требования ОС — от 20 до 40 ГБ в зависимости от дистрибутива Linux.

Рекомендуемый общий объём: от 32 ГБ для комфортной работы.

2.5 Сетевые требования

Для работы в локальной сети высоких требований к пропускной способности канала нет, так как происходит лишь передача текстовой информации (входящие запросы и исходящие ответы). Средний размер запроса/ответа не превышает нескольких килобайт, поэтому даже стандартного Gigabit Ethernet (1 Гбит/с) достаточно для обслуживания десятков пользователей.

Для упрощения настройки рекомендуется статический IP-адрес сервера. Сервер привязывается к порту (по умолчанию 8080) и сетевому интерфейсу 0.0.0.0 для обеспечения доступности из локальной сети.

2.6 Итоговые минимальные требования для реальной эксплуатации

На основе проведённого анализа, для сервера, обслуживающего 5-10 разработчиков с моделью 7В-13В параметров, аппаратные требования представлены в таблице 1:

Таблица 1: Аппаратные требования к серверу

Компонент	Минимальные требования	Рекомендуемые требования
CPU	8 ядер	16 ядер
RAM	16 ГБ	64 ГБ
GPU (опционально)	NVIDIA с 8 ГБ VRAM	NVIDIA с 16 ГБ VRAM
Накопитель	64 ГБ SSD	128 ГБ SSD
Сеть	Gigabit Ethernet	Gigabit Ethernet

2.7 Конфигурация для курсовой работы

Для демонстрационных целей и отработки навыков развёртывания в рамках курсовой работы я выбрал упрощенную конфигурацию на личном компьютере. Данная конфигурация не удовлетворяет требованиям к реальной многопользовательской среде, но позволяет полностью пройти цикл установки и настройки Llama-server; освоить конфигурирование параметров запуска; протестировать интеграцию со средой разработки; проверить работоспособность связки инструментов.

Для экспериментов была выбрана квантованная модель Qwen2.5-Coder-1.5B в формате GGUF с квантованием Q8_0. Выбор модели определялся следующими факторами:

- 1) Малый размер файла модели (~1.8 ГБ), что позволяет загрузить её на ограниченном оборудовании.
- 2) Достаточное качество ответов на русском языке для демонстрации работы.
- 3) Возможность запуска как на CPU, так и с частичной загрузкой слоёв на GPU.

3 РАЗРАБОТКА КОНФИГУРАЦИИ СЕРВЕРА

Серверная платформа была разработана на Arch Linu. Выбор конкретного дистрибутива Linux не имеет значения. Llama.cpp компилируется из исходного кода и будет работать любом дистрибутиве с поддержкой компилятора g++. Модель запускалась на процессоре, а также часть слоев на GPU.

3.1 Подготовка программного окружения

Llama.cpp собирается из исходного кода, из репозитория проекта на GitHub. Компиляция производилась прямо на компьютере при помощи CMake, что делает исполняемые файлы оптимизированными для конкретного процессора.

Для работы на ограниченном оборудовании была выбрана квантованная модель Qwen2.5-coder в формате GGUF с квантованием Q8_0. Выбор модели определялся её малым размером (~1,8ГБ) и при этом достаточным качеством ответов на русском языке. Модель была размещена в отдельной директории для удобства работы.

3.2 Скрипт запуска сервера

Для удобного запуска llama-server необходимо написать bash скрипт, скрипт на рисунке 1:

```

... > _ srv-conf.sh
1  #!/bin/bash
2
3  MODEL_PATH="/home/chitak/models/qwen2.5-coder-1.5b-instruct-q8_0.gguf"
4  HOST="0.0.0.0"
5  PORT="8080"
6
7  # Папка для логов
8  LOG_DIR="/home/chitak/Documents/курсовая/srv-logs"
9  mkdir -p "$LOG_DIR"
10
11 # Лог-файл с датой
12 LOG_FILE="$LOG_DIR/llama-server-$(date +%Y%m%d_%H%M%S).log"
13
14 # Получаем IP-адрес
15 IP=$(ip -4 addr show enp6s0 | grep -oP '(?<=inet\s)\d+(\.\d+){3}' | head -1)
16
17 # Запуск сервера
18 nohup /home/chitak/LLAMA/llama.cpp/build-pc/bin/llama-server \
19     -m "$MODEL_PATH" \
20     --host "$HOST" \
21     --port "$PORT" \
22     -c 4096 \
23     -ngl 33 \
24     --threads 4 \
25     --parallel 2 \
26     > "$LOG_FILE" 2>&1 &
27
28 PID=$!
29 echo $PID > /tmp/llama-server.pid
30
31 echo "Server start with PID: $PID"
32 echo "Web-interface: http://localhost:$PORT"
33 echo "Log: $LOG_FILE"
34 echo "For stop: kill $PID"
35

```

Рисунок 1 - Скрипт запуска сервера

Параметр «-c 4096» был выбран как компромиссное значение: для многих задач, контекста в 4096 токена достаточно. Параметр «-ngl 33» загружает 33 слоя модели в видеопамять и они обрабатываются GPU. Параметр «--threads 4» отвечает за количество ядер процессора отведенных под вычисления. Параметр «--parallel 2» позволяет серверу обрабатывать до двух запросов одновременно, что критически важно при интеграции с несколькими клиентами в локальной сети.

3.3 Работа модели

Для проверки работоспособности модели я на компьютере открыл в браузере веб интерфейс llama-srver по адресу «http://127.0.0.1:8080». В чате с моделью протестировал тестовый промт, результат на рисунке 2:

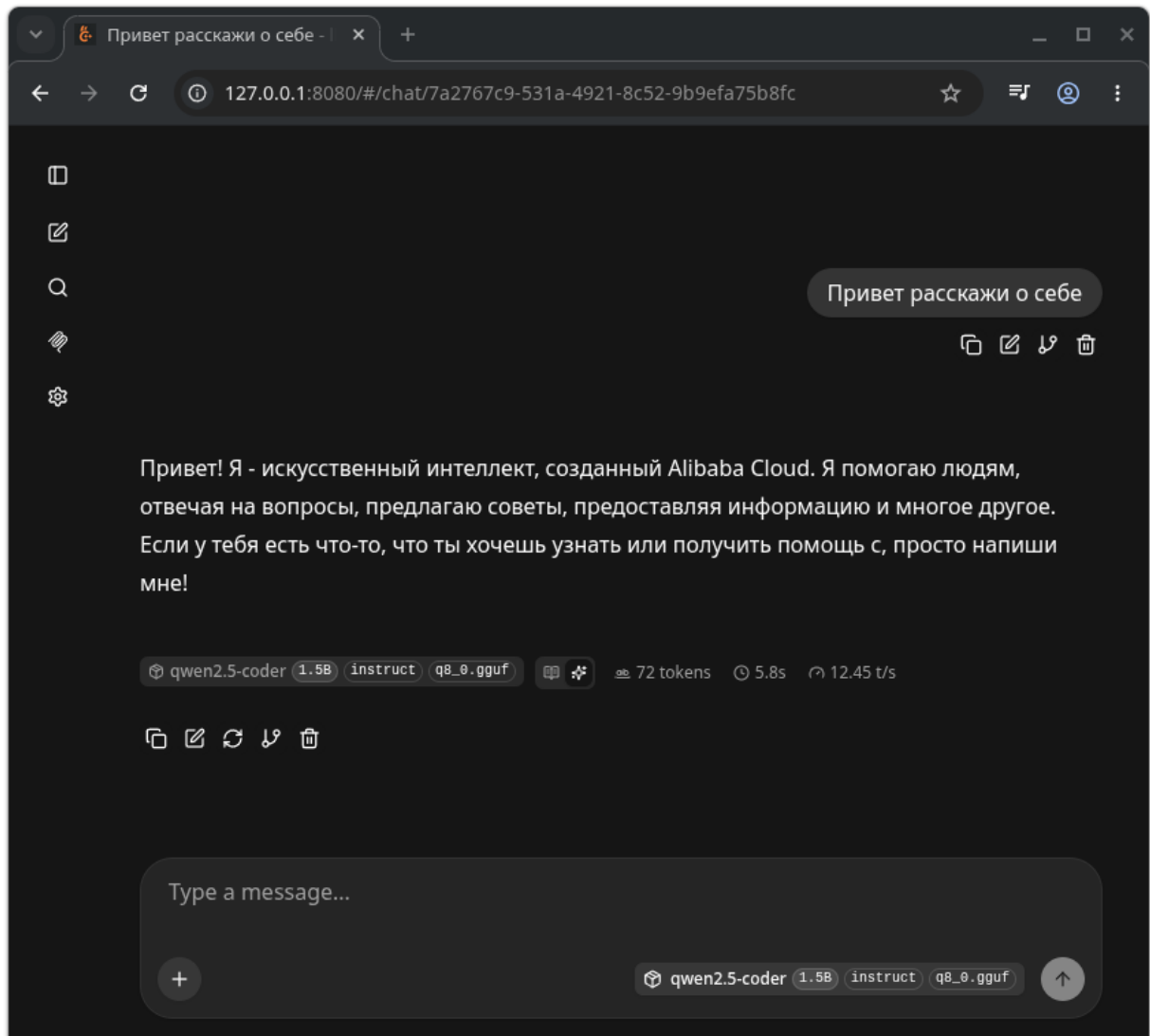


Рисунок 2 - Вэб интерфейс сервера

Модель успешно дала ответ на запрос, что свидетельствует о правильной работе llama-server.

4 РЕАЛИЗАЦИЯ ИНТЕГРАЦИИ ЛОКАЛЬНОГО LLM-СЕРВЕРА СО СРЕДОЙ РАЗРАБОТКИ CODE OSS

4.1 Выбор инструмента интеграции: расширение Continue

Для интеграции разработанного локального LLM-сервера в среду разработки был выбран инструмент Continue — открытое расширение (плагин) для Visual Studio Code и его форков, включая Code OSS. Continue позиционируется как AI-ассистент для разработчиков, поддерживающий подключение к различным моделям, включая локальные

Важно отметить, что Continue доступен не только в официальном Visual Studio Code от Microsoft, но и в Code OSS — полностью открытой версии редактора, распространяемой под лицензией MIT. Это позволяет использовать расширение в среде разработки, где приоритетом является полная открытость и отсутствие телеметрии Microsoft.

Continue использует конфигурационный файл в формате YAML, который по умолчанию находится в директории `~/.continue/config.yaml` (в Linux-системах). Для подключения к локальному серверу на базе llama.cpp была создана следующая конфигурация, на рисунке 3:

```

home > chitak > .continue > ! config.yaml
View Continue Reference | Download YAML extension for Intellisense
1  name: Local Config
2  version: 1.0.0
3  schema: v1
4
5  models:
6    - name: Local llama-server
7      provider: llama.cpp
8      model: qwen2.5-coder-1.5b-instruct-q8_0.gguf # имя модели
9      apiBase: http://127.0.0.1:8080 # адрес сервера
10     contextLength: 8192
11     roles:
12       - chat
13       - edit
14       - apply
15
16   tabAutocompleteModel:
17     provider: llama.cpp
18     model: qwen2.5-coder-1.5b-instruct-q8_0.gguf
19     apiBase: http://127.0.0.1:8080
20     contextLength: 4096
21
22   allowAnonymousTelemetry: false

```

Рисунок 3 - Конфигурация плагина Continue

4.2 Проверка работоспособности интеграции

После сохранения конфигурации и перезагрузки Code OSS расширение Continue установило соединение с llama-server. Успешность подключения подтверждалась появлением имени модели «Local llama-server» в интерфейсе Continue (нижняя панель чата).

Для проверки качества работы модели был отправлен запрос на написание программы для рисования квадрата из выбранного символа. Результат на рисунке 4.

2 diffs

напиши программу для рисования квадрата из указанного символа

Local llama-server

home > chitak > Documents > курсовая > test_code > test1.cpp

Accept | Reject

```

1  #include <iostream>
2  using namespace std;
3  int main() {
4      // Prompt user to enter the character for the square
5      char symbol;
6      cout << "Enter the character for the square: ";
7      cin >> symbol;
8
9      // Prompt user to enter the size of the square
10     int size;
11     cout << "Enter the size of the square: ";
12     cin >> size;
13
14     // Print the top and bottom rows of the square
15     for (int i = 0; i < size; i++) {
16         cout << symbol;
17     }
18     cout << endl;
19
20     // Print the middle rows of the square
21     for (int i = 0; i < size - 2; i++) {
22         cout << symbol;
23         for (int j = 1; j < size - 1; j++) {
24             cout << " ";
25         }
26         cout << symbol;
27         cout << endl;
28     }
29
30     // Print the last row of the square
31     for (int i = 0; i < size; i++) {
32         cout << symbol;
33     }
34     cout << endl;
35
36     return 0;
37 }
```

Рисунок 4 - Код сгенерированный моделью

Код успешно компилируется и работает, показано на рисунке 5:

```

chitak@X99:~/Documents/курсовая/test_code$ g++ -Wall -Wextra test1.cpp -o test1-bin
chitak@X99:~/Documents/курсовая/test_code$ ./test1-bin
Enter the character for the square: #
Enter the size of the square: 16
#####
#               #
#               #
#               #
#               #
#               #
#               #
#               #
#               #
#               #
#               #
#               #
#               #
#               #
#####
chitak@X99:~/Documents/курсовая/test_code$ █

```

Рисунок 5 - Компиляция и работа программы

4.3 Выводы по интеграции

Интеграция локального LLM-сервера с Code OSS через расширение Continue была успешно реализована на уровне подключения: клиент (Code OSS) установил соединение с сервером (llama-server) в локальной сети и смог отправлять запросы на генерацию. Однако практическая польза данной конфигурации мала из-за слабого оборудования и маленькой модели.

Данный результат, тем не менее, имеет ценность: он демонстрирует принципиальную достижимость цели — локальный LLM-сервер может быть интегрирован в среду разработки, заменяя облачные решения. Для повышения качества генерации потребуется использование более крупных моделей, например Qwen3.6-27B на 27 млрд параметров, что, в свою очередь, потребует более мощного серверного оборудования.

ЗАКЛЮЧЕНИЕ

Цель курсовой работы — создание локального LLM сервера, его администрирование и интеграция в рабочую среду была успешно выполнена. Для ее выполнения понадобились навыки, знания и опыт, полученные в ходе обучения и практик, а так же большое количество самостоятельного обучения

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

<https://stackoverflow.co/company/press/archive/stack-overflow-2025-developer-survey>

<https://www.rdworldonline.com/how-developers-and-engineers-are-learning-to-work-with-ai-they-dont-fully-trust>

<https://neuralnoise.com///2026/harness-bench-wip/>